ReqRes REST API Specification & Documentation

Complete Endpoint Reference, Request/Response Payloads & QA Test Matrix

Target Base URL: https://reqres.in **API Format:** RESTful JSON **Author / SDET:** Tanzim Rahman (TanzimSQA)

Repository: github.com/TanzimSQA/reqres-postman-api-testing

Test Suite: Postman Collection v2.1.0 (18 Requests / 80 Assertions)

---

1. Overview & General Specifications

This document serves as the formal technical documentation for the **ReqRes REST API** (`https://reqres.in`) and its accompanying automated regression test suite. ReqRes is a mock RESTful web service providing realistic responses for prototyping and QA test automation.

Key Architectural Principles

- **Protocol:** HTTPS only (port 443).
- **Content Negotiation:** All requests sending a body must include `Content-Type: application/json`. All responses return `application/json; charset=utf-8`.
- **Authentication:** Mock token authentication returning session tokens for valid registered users.
- **Rate Limiting:** Enforces throttling (HTTP 429) on public shared IP pools. Test scripts incorporate rate-limit resilience.

2. Complete API Endpoint Reference

Module 1: User Management (CRUD)

`GET` **/api/users?page={page}** — 200 OK

Retrieves a paginated list of user records.

Query Parameters:

- page (number, optional, default: 1): Page index
- per_page (number, optional, default: 6): Items per page

Sample Response (200 OK):


```
{
  "page": 1,
  "per_page": 6,
  "total": 12,
  "total_pages": 2,
  "data": [
    {
      "id": 1,
      "email": "george.bluth@reqres.in",
      "first_name": "George",
      "last_name": "Bluth",
      "avatar": "https://reqres.in/img/faces/1-image.jpg"
    }
  ]
}
```

GET /api/users/{id} 200 OK / 404 Not Found

Fetches full profile information for a specific user ID.

Path Parameters:

- `id` (number, required): User ID (e.g., 2)

Response Codes:

- `200 OK`: Valid user record returned
- `404 Not Found`: Non-existent ID (returns `{}`)

Sample Response (200 OK):


```
{
  "data": {
    "id": 2,
    "email": "janet.weaver@reqres.in",
    "first_name": "Janet",
    "last_name": "Weaver",
    "avatar": "https://reqres.in/img/faces/2-image.jpg"
  },
  "support": {
    "url": "https://contentcaddy.io?utm_source=reqres&utm_medium=json&utm_campaign=refe
    "text": "Tired of writing mock data?..."
  }
}
```


POST /api/users 201 Created

Creates a new user record and returns the allocated ID with timestamp.

Request Body (JSON):


```
{
  "name": "Tanzim SQA",
  "job": "Lead QA Automation Engineer"
}
```


Sample Response (201 Created):


```
{
  "name": "Tanzim SQA",
  "job": "Lead QA Automation Engineer",
  "id": "416",
  "createdAt": "2026-10-16T09:14:21.054Z"
}
```

PUT /api/users/{id} 200 OK

Performs a full replacement update on a user record.

Request Body (JSON):

```
{
  "name": "Tanzim SQA - Updated",
  "job": "Principal QA Automation Architect"
}
```

Sample Response (200 OK):

```
{
  "name": "Tanzim SQA - Updated",
  "job": "Principal QA Automation Architect",
  "updatedAt": "2026-10-16T09:14:22.100Z"
}
```

PATCH /api/users/{id} 200 OK

Performs a partial update (e.g. updating job title only).

Request Body (JSON):

```
{
  "job": "Staff SDET"
}
```

Sample Response (200 OK):

```
{
  "job": "Staff SDET",
  "updatedAt": "2026-10-16T09:14:23.010Z"
}
```

DELETE /api/users/{id} 204 No Content

Deletes an existing user record. Returns HTTP 204 with an empty response body.

Module 2: Resources (Colors Catalog)

GET /api/unknown 200 OK

Returns list of color resources including hex code and Pantone values.

```
{
  "page": 1,
  "per_page": 6,
  "total": 12,
  "data": [
    {
      "id": 1,
      "name": "cerulean",
      "year": 2000,
      "color": "#98B2D1",
      "pantone_value": "15-4020"
    }
  ]
}
```

Module 3: Authentication & Authorization

`POST` /api/register — 200 OK / 400 Bad Request

Valid Request Body:

```
{
  "email": "eve.holt@reqres.in",
  "password": "pistol"
}
```

Success Response (200 OK):

```
{
  "id": 4,
  "token": "QpwL5tke4Pnpja7X4"
}
```

Missing Password Request:

```
{
  "email": "sydney@fife"
}
```

Error Response (400 Bad Request):

```
{
  "error": "Missing password"
}
```

POST /api/login — 200 OK / 400 Bad Request

Valid Login Request:

```
{
  "email": "eve.holt@reqres.in",
  "password": "cityslicka"
}
```

Success Response (200 OK):

```
{
  "token": "QpwL5tke4Pnpja7X4"
}
```

Invalid Login Scenarios (400 Bad Request):

- Missing password: `{"error": "Missing password"}`
- Unregistered user: `{"error": "user not found"}`

Module 4: Performance & Network Latency

GET /api/users?delay={seconds} — 200 OK

Simulates backend network latency by delaying response delivery by the requested duration in seconds. Tests client timeouts and payload preservation under latency.

3. Automated Test Suite Matrix (18 Tests, 80 Assertions)

#	Endpoint	Method	Status	Key Automated Assertions & Business Rules
1.1	/api/users?page=1	GET	200	Page count, user array length, email regex (/@reqres\.in$/), avatar URL
1.2	/api/users?page=2	GET	200	Unique ID check; dynamically extracts first ID -> selectedUserId
1.3	/api/users/{{selectedUserId}}	GET	200	Returned user ID matches {{selectedUserId}}, non-empty profile values
1.4	/api/users/23	GET	404	Negative test: Non-existent ID returns empty object {}
1.5	/api/users	POST	201	Generates unique timestamp name; extracts returned id -> createdUserId
1.6	/api/users/{{createdUserId}}	PUT	200	Full entity replacement; verifies name, job, and updatedAt timestamp
1.7	/api/users/{{createdUserId}}	PATCH	200	Partial update; verifies updated job title and timestamp
1.8	/api/users/{{createdUserId}}	DELETE	204	Guarantees HTTP 204 No Content and empty response body
2.1	/api/unknown	GET	200	Hex color format (^#[0-9A-Fa-f]{6}$) & Pantone pattern (^[0-9]{2}-[0-9]{4}$)
2.2	/api/unknown/2	GET	200	Resource ID 2 matches 'fuchsia rose', year 2001, #C74375
2.3	/api/unknown/23	GET	404	Negative test: Invalid color resource returns 404 and empty body
3.1	/api/register	POST	200	Valid registration; extracts token -> authToken
3.2	/api/register	POST	400	Negative test: Missing password payload; asserts 'Missing password'
3.3	/api/register	POST	400	Negative test: Missing email payload; asserts 'Missing email or username'
3.4	/api/login	POST	200	Valid login; extracts session token -> loginToken
3.5	/api/login	POST	400	Negative test: Missing password login; asserts 400 error message
3.6	/api/login	POST	400	Negative test: Unregistered user login; asserts 'user not found'
4.1	/api/users?delay=1	GET	200	Latency SLA verification (< 5000ms) and payload preservation

4. Test Execution & CI/CD Guide

```
# 1. Run full test suite via Newman CLI
npx newman run ReqRes_API_Testing.postman_collection.json \
  -e ReqRes_Environment.postman_environment.json \
  --delay-request 500

# 2. Automated GitHub Actions Workflow (.github/workflows/newman-api-tests.yml)
Runs on every push to main; uploads rich HTML report artifact on completion.
```